

SAP BTP Monitor

Documentation Technique Complète

Un outil de monitoring SAP BTP construit en Rust ®

Version 0.4.0 • Avril 2025

développé par : Carsouille Dawson

⚠ **Projet en cours de développement actif**



**The Rust
Programming
Language**

Table des Matières

Table des Matières.....	2
1. Introduction et Vue d'Ensemble.....	5
1.1 Qu'est-ce que SAP BTP Monitor ?.....	5
1.2 Statut du projet.....	5
1.3 Architecture globale.....	5
2. Rust — Présentation et Installation.....	7
2.1 Pourquoi Rust ?.....	7
Rust vs C — Les avantages clés.....	7
2.2 Installation de Rust.....	8
Linux / macOS.....	8
Windows.....	8
Vérification de l'installation.....	8
2.3 Premier programme Rust — Hello, World!.....	8
Création manuelle.....	8
Avec Cargo (approche recommandée).....	9
2.4 L'écosystème Cargo et Crates.io.....	9
3. Installation et Configuration du Projet.....	11
3.1 Prérequis.....	11
3.2 Récupération du code source.....	11
3.3 Configuration de l'environnement (.env).....	12
3.4 Préparation de la base de données PostgreSQL.....	12
4. Compilation et Exécution.....	15
4.1 Compilation.....	15
4.2 Modes d'exécution.....	15
4.3 Arguments CLI disponibles.....	15
4.4 Configuration persistante.....	16
4.5 Vérification de santé (health check).....	16
5. Description des Fichiers Source.....	17
5.1 main.rs — Point d'entrée et Orchestration.....	17
Responsabilités principales.....	17
Fonctions clés.....	17
Concurrence Tokio.....	17
5.2 api.rs — Client HTTP SAP.....	18
Fonctions clés.....	18
Parsing des dates SAP.....	18
5.3 config.rs — Configuration.....	18
SapConfig.....	19
TokenCache.....	19
UserConfig.....	19
5.4 crypto.rs — Chiffrement AES-256-GCM.....	19
5.5 models.rs — Modèles de Données.....	20
Structures principales.....	20
5.6 db.rs — Accès à la Base de Données.....	20
Pattern d'insertion — ON CONFLICT.....	21
Fonctions clés.....	21
5.7 queries.rs — Requêtes Agrégées.....	21
Requêtes de visualisation.....	22
5.8 webhook.rs — Alertes Microsoft Teams.....	22

Flux de traitement.....	22
Format des alertes.....	22
5.9 ui.rs — Interface TUI Ratatui.....	23
5.10 utils.rs — Utilitaires.....	23
6. Guide de l'Interface TUI.....	24
6.1 Présentation générale.....	24
6.2 Navigation entre onglets.....	25
6.3 Navigation dans les listes.....	26
6.4 Onglet Logs.....	26
Colonnes affichées.....	26
Panneau de détail.....	26
6.5 Onglet Artifacts.....	27
Colonnes affichées.....	28
6.6 Onglet Packages.....	29
Niveaux de navigation.....	30
6.7 Onglets Erreurs.....	30
Err.Déploi. — Erreurs de Déploiement.....	30
Err.Exéc. — Erreurs d'Exécution.....	31
6.8 Onglet Analytics.....	32
6.9 Recherche et Filtres.....	33
Recherche textuelle.....	33
Filtres structurés (key:value).....	34
Recherche libre (full-text).....	34
filtre par tenant.....	35
Filtre par date (Calendrier interactif).....	36
6.10 Rafraîchissement des données.....	38
7. Dépendances du Projet.....	39
8. Référence Rapide.....	41
8.1 Raccourcis clavier — Résumé complet.....	41
8.2 Variables d'environnement.....	42
8.3 Commandes Cargo essentielles.....	42
9. Résolution des Problèmes Courants.....	43
9.1 Erreurs de compilation.....	43
9.2 Erreurs d'exécution.....	43
9.3 Performances (encore en optimisation).....	43
10. Roadmap et Évolutions Prévues.....	44
10.1 Fonctionnalités prioritaires.....	44
10.2 Améliorations TUI.....	44
10.3 Infrastructure.....	44
Glossaire.....	45

1. Introduction et Vue d'Ensemble

1.1 Qu'est-ce que SAP BTP Monitor ?

SAP BTP Monitor est un outil de supervision en ligne de commande (TUI — Terminal User Interface) dédié à la plateforme SAP Business Technology Platform (SAP BTP). Il permet aux équipes techniques de surveiller en temps réel l'état des intégrations SAP Cloud Integration (CPI) directement depuis un terminal, sans navigateur.

L'outil centralise trois grandes responsabilités :

- **Monitoring des logs d'exécution** (*MessageProcessingLogs*) : visualise les statuts COMPLETED, FAILED, PROCESSING de tous les flux d'intégration.
- **Inventaire des artifacts et packages** : liste les IntegrationPackages et RuntimeArtifacts déployés, avec leurs configurations externalisées.
- **Alertes webhook Teams** : envoie des notifications groupées (Adaptive Cards) vers Microsoft Teams lorsque des erreurs sont détectées.

1.2 Statut du projet

⚠️ Projet en cours de développement actif

SAP BTP Monitor est en version 0.3.0. Certaines fonctionnalités sont encore en développement ou susceptibles d'évoluer. L'interface TUI est fonctionnelle mais continue d'être enrichie. Les tests automatisés et la couverture de code ne sont pas encore formalisés.

1.3 Architecture globale

Le projet suit une architecture en couches clairement séparées :

Couche	Responsabilité	Fichier(s)
Configuration	Chargement des variables d'environnement et persistance de la config utilisateur	config.rs
Modèles	Structures de données Serde pour la désérialisation JSON SAP OData	models.rs
API Client	Appels HTTP OAuth + endpoints SAP Cloud Integration	api.rs

Base de données	Persistance PostgreSQL via SQLx (inserts, queries)	db.rs
Requêtes	Construction du payload de rafraîchissement (agrégats, statistiques)	queries.rs
Interface TUI	Rendu Ratatui + gestion des événements clavier	ui.rs
Webhooks	Envoi d'alertes Teams Adaptive Cards avec anti-spam	webhook.rs
Utilitaires	Spinner de progression CLI	utils.rs
Point d'entrée	Orchestration async, workers Tokio, boucle principale	main.rs

2. Rust — Présentation et Installation

2.1 Pourquoi Rust ?

Rust est un langage de programmation systèmes créé par Mozilla Research (2010) et aujourd'hui géré par la Rust Foundation. Il est conçu pour offrir les performances du C/C++ tout en garantissant la sécurité mémoire sans ramasse-miettes (garbage collector).

Dans le contexte de SAP BTP Monitor, Rust a été choisi pour plusieurs raisons techniques :

Critère	Rust	Python (alternatif)
Performances	Comparable au C — zéro overhead à l'exécution	Interprété — 10x à 100x plus lent
Sécurité mémoire	Garantie à la compilation (ownership/borrowing)	Runtime (GC, risques de fuite)
Concurrence	async/await natif, sans data races garanties	GIL limitant, complexité manuelle
Binaire final	Exécutable unique, aucune dépendance runtime	Nécessite Python + environnement
TUI	Ratatui — bibliothèque dédiée, très performante	Cursors/Rich — moins optimisés

Rust vs C — Les avantages clés

Le C est un langage puissant mais dangereux : il laisse entièrement au développeur la responsabilité de la gestion mémoire. Les erreurs les plus courantes en C — dépassement de tampon (buffer overflow), utilisation après libération (use-after-free), pointeurs nuls — sont responsables de la majorité des failles de sécurité critiques dans les systèmes modernes.

Rust résout ces problèmes fondamentalement différemment, à la compilation :

- **Ownership** : chaque valeur a un propriétaire unique. Quand le propriétaire sort de portée, la mémoire est libérée automatiquement — sans GC.
- **Borrowing** : des références (emprunts) peuvent être créées avec des règles strictes : une seule référence mutable à la fois, ou plusieurs références immuables.
- **Lifetimes** : le compilateur vérifie que les références ne survivent jamais à la donnée qu'elles pointent — impossible d'avoir un dangling pointer.
- **Pas de null** : le type `Option<T>` force la gestion explicite des valeurs absentes — les `NullPointerException` n'existent pas.

Le principe fondateur de Rust

"Si ça compile, ça ne crashe pas en production."

Le compilateur Rust (rustc) est à la fois votre pair reviewer et votre garde-fou. Il refuse de compiler du code qui présente des risques de corruption mémoire ou de data race. Ce qui en C produirait un crash à l'exécution devient une erreur de compilation en Rust.

2.2 Installation de Rust

Rust s'installe via rustup, l'outil officiel de gestion des toolchains. Il gère l'installation du compilateur (rustc), du gestionnaire de paquets (cargo) et des mises à jour.

Linux / macOS

```
# Installation via rustup (méthode officielle recommandée)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Suivre les instructions à l'écran, puis recharger le shell
source $HOME/.cargo/env

# Vérifier l'installation
rustc --version
cargo --version
```

Windows

Sur Windows, télécharger et exécuter [rustup-init.exe](https://rustup.rs) depuis le site officiel <https://rustup.rs>. Microsoft Visual C++ Build Tools est également nécessaire (inclus avec Visual Studio ou téléchargeable séparément).

Vérification de l'installation

```
rustc --version # ex: rustc 1.78.0 (9b00956e5 2024-04-29)
cargo --version # ex: cargo 1.78.0 (54d8815d0 2024-03-26)
rustup show     # affiche les toolchains installées
```

2.3 Premier programme Rust — Hello, World!

Voici le programme traditionnel de découverte d'un nouveau langage :

Création manuelle

```
# Créer un fichier main.rs
fn main() {
    println!("Hello, World!");
}
```

```
# Compiler et exécuter
rustc main.rs
./main
```

Avec Cargo (approche recommandée)

Cargo est le gestionnaire de projet officiel de Rust. Il gère la compilation, les dépendances, les tests et la documentation.

```
# Créer un nouveau projet
cargo new hello-world
cd hello-world

# Structure créée automatiquement :
# hello-world/
# └─ Cargo.toml    ← Manifeste du projet (dépendances, métadonnées)
# └─ src/
#     └─ main.rs   ← Point d'entrée

# Contenu de src/main.rs (généré automatiquement) :
fn main() {
    println!("Hello, World!");
}

# Compiler et exécuter en une commande
cargo run

# Compiler en mode release (optimisé, plus lent à compiler)
cargo build --release

# Compiler et exécuter en version optimisé
cargo run --release
```

Concepts clés du Hello World Rust

`fn main()` : Point d'entrée du programme, équivalent au `int main()` du C.

`println!` : Macro (notez le !) qui affiche une ligne. Les macros en Rust sont une forme de métaprogrammation hygiénique.

`Cargo.toml` : Fichier TOML (Tom's Obvious Minimal Language) qui liste les dépendances et la configuration du projet — équivalent du `package.json` de Node ou du `requirements.txt` de Python.

2.4 L'écosystème Cargo et Crates.io

Les bibliothèques Rust s'appellent des **crates**. Elles sont publiées sur crates.io, le registre officiel. On les déclare dans `Cargo.toml` sous `[dependencies]` :

```
[dependencies]
tokio = { version = "1", features = ["full"] }
serde = { version = "1.0", features = ["derive"] }
sqlx = { version = "0.7", features = ["runtime-tokio-native-tls", "postgres"] }
```

Cargo télécharge, compile et lie automatiquement toutes les dépendances et leurs dépendances transitives lors du premier build.

3. Installation et Configuration du Projet

3.1 Prérequis

Composant	Version minimale	Usage
Rust + Cargo	1.75+	Compilation du projet
PostgreSQL	14+	Base de données de persistance
Compte SAP BTP	—	Accès à l'API Cloud Integration
Client OAuth SAP	—	Credentials client_id / client_secret
Microsoft Teams (optionnel)	—	Webhook pour les alertes

3.2 Récupération du code source

```
# Cloner le dépôt
git clone https://github.com/dawsoncarsouille-lab/cerealog-rust.git sap-btp-monitor
cd sap-btp-monitor

# Structure du projet
sap-btp-monitor/
├── Cargo.toml      ← Manifeste Cargo
├── src/
│   ├── main.rs    ← Point d'entrée + orchestration
│   ├── api.rs     ← Client HTTP SAP
│   ├── config.rs  ← Configuration + cache OAuth
│   ├── db.rs      ← Accès PostgreSQL
│   ├── models.rs  ← Structures de données
│   ├── queries.rs ← Requêtes agrégées
│   ├── /ui        ← Interface TUI Ratatui
│   ├── webhook.rs ← Alertes Teams
│   └── utils.rs   ← Utilitaires CLI
├── .env.exemple   ← Template de configuration
└── table.sql      ← fichier des tables utilisés
```

3.3 Configuration de l'environnement (.env)

Créer un fichier `.env` à la racine du projet, basé sur le template fourni :

```
# Clé pour crypter client_secret
TENANT_ENCRYPTION_KEY='' # en hexa sur 32 bits

# Connection string PostgreSQL
DATABASE_URL=postgresql://user:password@localhost:5432/sap_monitor

# (Optionnel) URL webhook Microsoft Teams pour les alertes
WEBHOOK_URL=https://outlook.office.com/webhook/...
```

Sécurité — Ne jamais committer le .env

Le fichier `.env` contient des credentials sensibles. Il doit impérativement figurer dans `.gitignore`. En production, préférer les variables d'environnement système ou un gestionnaire de secrets (HashiCorp Vault, AWS Secrets Manager, etc.).

3.4 Préparation de la base de données PostgreSQL

Le projet utilise PostgreSQL comme base de persistance. Les tables sont créées manuellement (les migrations SQLx ne sont pas encore automatisées dans cette version).

```
CREATE TABLE IF NOT EXISTS integration_packages (
    id VARCHAR(255) PRIMARY KEY,
    name TEXT,
    version VARCHAR(50),
    vendor TEXT,
    creation_date TIMESTAMP,
    tags TEXT
);

CREATE TABLE IF NOT EXISTS runtime_artifacts (
    id VARCHAR(255) PRIMARY KEY,
    name TEXT,
    status VARCHAR(50),
    deployed_on TIMESTAMP,
    package_id VARCHAR(255) REFERENCES integration_packages(id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS sap_monitoring_logs (
    message_guid VARCHAR(255) PRIMARY KEY,
    status VARCHAR(50),
    parsed_date TIMESTAMP,
    error_message TEXT,
```

```

        integration_flow_name TEXT
    );

CREATE TABLE IF NOT EXISTS artifact_errors (
    artifact_id VARCHAR(255) PRIMARY KEY REFERENCES runtime_artifacts(id) ON DELETE
    CASCADE,
    error_message TEXT,
    error_time TIMESTAMP
);

CREATE TABLE IF NOT EXISTS artifact_configurations (
    id SERIAL PRIMARY KEY,
    artifact_id VARCHAR(255) NOT NULL REFERENCES runtime_artifacts(id) ON DELETE
    CASCADE,
    parameter_key TEXT NOT NULL,
    parameter_value TEXT,
    data_type VARCHAR(50),
    UNIQUE(artifact_id, parameter_key)
);

CREATE INDEX IF NOT EXISTS idx_logs_date ON sap_monitoring_logs (parsed_date DESC);
CREATE INDEX IF NOT EXISTS idx_logs_status_date ON sap_monitoring_logs (status,
    parsed_date);
CREATE INDEX IF NOT EXISTS idx_logs_flow_name ON sap_monitoring_logs
    (integration_flow_name);
CREATE INDEX IF NOT EXISTS idx_artifacts_package ON runtime_artifacts (package_id);

ALTER TABLE integration_packages ADD COLUMN tenant_id VARCHAR(100) NOT NULL DEFAULT
    'cerealog';
ALTER TABLE runtime_artifacts ADD COLUMN tenant_id VARCHAR(100) NOT NULL DEFAULT
    'cerealog';
ALTER TABLE sap_monitoring_logs ADD COLUMN tenant_id VARCHAR(100) NOT NULL DEFAULT
    'cerealog';
ALTER TABLE artifact_errors ADD COLUMN tenant_id VARCHAR(100) NOT NULL DEFAULT
    'cerealog';
ALTER TABLE artifact_configurations ADD COLUMN tenant_id VARCHAR(100) NOT NULL
    DEFAULT 'cerealog';

CREATE TABLE IF NOT EXISTS tenants (
    id VARCHAR(100) PRIMARY KEY,
    name TEXT NOT NULL,
    client_name TEXT,
    shared_tenant BOOLEAN DEFAULT false,
    sap_base_url TEXT,
    sap_token_url TEXT,
    active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT NOW()
);

INSERT INTO tenants (id, name, client_name, shared_tenant, active)
VALUES ('cerealog', 'Cerealog', 'Cerealog', false, true)
ON CONFLICT (id) DO NOTHING;

CREATE INDEX IF NOT EXISTS idx_packages_tenant ON integration_packages (tenant_id);
CREATE INDEX IF NOT EXISTS idx_artifacts_tenant ON runtime_artifacts (tenant_id);
CREATE INDEX IF NOT EXISTS idx_logs_tenant ON sap_monitoring_logs (tenant_id);

ALTER TABLE integration_packages ADD CONSTRAINT fk_packages_tenant FOREIGN KEY
    (tenant_id) REFERENCES tenants(id);
ALTER TABLE runtime_artifacts ADD CONSTRAINT fk_artifacts_tenant FOREIGN KEY
    (tenant_id) REFERENCES tenants(id);

```

```
ALTER TABLE sap_monitoring_logs ADD CONSTRAINT fk_logs_tenant FOREIGN KEY
(tenant_id) REFERENCES tenants(id);

ALTER TABLE tenants ADD COLUMN sap_client_id TEXT;
ALTER TABLE tenants ADD COLUMN sap_client_secret_enc TEXT;

DROP TABLE IF EXISTS smart_alerts;

CREATE TABLE smart_alerts (
    id                BIGSERIAL PRIMARY KEY,
    tenant_id         VARCHAR(100) NOT NULL,
    flow_name         TEXT NOT NULL,
    alert_type        TEXT NOT NULL,
    status            TEXT NOT NULL DEFAULT 'PENDING',
    last_triggered_at  TIMESTAMP NOT NULL DEFAULT NOW(),
    extra             JSONB,
    CONSTRAINT smart_alerts_unique UNIQUE (tenant_id, flow_name, alert_type)
);
```

4. Compilation et Exécution

4.1 Compilation

```
# Mode développement (rapide, avec symboles de débogage)
cargo build

# Mode release (optimisé, binaire plus petit et plus rapide)
cargo build --release

# Le binaire compilé se trouve dans :
# target/debug/sap-extractor      (mode debug)
# target/release/sap-extractor    (mode release)
```

La compilation initiale peut prendre quelques minutes car Cargo télécharge et compile toutes les dépendances. Les compilations suivantes sont incrémentales et beaucoup plus rapides.

4.2 Modes d'exécution

```
# Lancer directement avec Cargo (mode développement)
cargo run # rajoutez --release pour avoir la version optimisé

# Lancer avec des arguments
cargo run -- --top 500
cargo run -- --full
cargo run -- --health

# Lancer le binaire release directement
./target/release/sap-extractor
./target/release/sap-extractor --top 1000 --full
```

4.3 Arguments CLI disponibles

Argument	Type	Description
--top <N>	u32	Nombre maximum de logs à récupérer depuis l'API SAP (défaut : 200, configurable dans la config utilisateur)
--full	flag	Force un fetch complet (packages + artifacts + logs) même si des données existent déjà en base. Sans ce flag, le fetch est incrémental (uniquement les nouveaux logs depuis le dernier fetch).
--health	flag	Mode diagnostic : teste

		uniquement la connexion SAP (OAuth) et la base de données, affiche un rapport, puis quitte.
--benchmark	flag	Mode benchmark (désactivé dans la version actuelle — réservé pour les tests de performance futurs). (désactivé dans cette version)

4.4 Configuration persistante

L'application sauvegarde automatiquement les préférences utilisateur dans un fichier TOML stocké dans le répertoire de configuration système de l'OS (via la crate `dirs`) :

```
# Linux
~/.config/sap-extractor/config.toml

# macOS
~/Library/Application Support/sap-extractor/config.toml

# Windows
C:\Users\<user>\AppData\Roaming\sap-extractor\config.toml

# Contenu exemple
logs_limit = 500
parallel_requests = 20
```

Le fichier stocke également le token OAuth en cache (avec sa date d'expiration) pour éviter une authentification à chaque démarrage. Le token est automatiquement renouvelé 120 secondes avant son expiration.

4.5 Vérification de santé (health check)

```
cargo run -- --health

# Sortie attendue :
❯ Vérification des connexions...
✓ SAP OK ✓ DB OK
```

5. Description des Fichiers Source

Cette section décrit en détail le rôle et le contenu de chaque fichier du projet.

5.1 main.rs — Point d'entrée et Orchestration

Rôle : Orchestration générale du programme. C'est le chef d'orchestre qui initialise toutes les ressources, lance les workers asynchrones et gère la boucle événementielle principale.

Responsabilités principales

- Parsing des arguments CLI via la crate Clap
- Chargement de la configuration (.env + UserConfig)
- Initialisation du pool de connexions PostgreSQL
- Création du canal async mpsc (worker → UI) pour passer les données
- Lancement du worker de fetch SAP (run_fetch_worker) dans une tâche Tokio
- Lancement du worker webhook
- Boucle principale : appel de run_tui() et traitement des événements retournés

Fonctions clés

sync_logs()	Récupère les logs depuis l'API SAP et les insère en base. Gère le mode incrémental (filtre depuis la date du dernier log connu).
sync_packages_and_artifacts()	Récupère en parallèle les packages et artifacts SAP, les insère en base, et lance les fetches d'erreurs pour les artifacts en erreur.
sync_configurations()	Pour chaque artifact connu, récupère ses configurations externalisées et met à jour la table artifact_configurations.
run_fetch_worker()	Orchestre l'ensemble du cycle de fetch. Gère le token OAuth, choisit entre fetch complet et incrémental, puis envoie les données à l'UI via le canal mpsc.
main()	Initialisation globale + boucle événementielle. Traite les AppEvent retournés par l'UI (Quit, TriggerRefresh, LoadMore, Continue).

Concurrence Tokio

Le programme utilise le runtime async Tokio avec un modèle multi-tâches. Les fetches HTTP sont lancés en parallèle avec des limites de concurrence configurable (50 pour les logs, 20 pour les packages/artifacts). Cela permet d'exploiter pleinement les connexions HTTP keep-alive tout en évitant de surcharger l'API SAP.

5.2 api.rs — Client HTTP SAP

Rôle : Toutes les communications avec l'API OData de SAP Cloud Integration. Authentification OAuth, fetch des données, gestion des erreurs HTTP.

Fonctions clés

build_http_client()	Construit un client request optimisé avec connection pooling (50 connexions idle max, keepalive TCP 60s). Ce client est réutilisé pour toute la durée d'exécution.
get_sap_token()	POST vers l'endpoint OAuth SAP avec Basic Auth (client_id:client_secret). Retourne un TokenCache avec TTL de 3500 secondes.
get_or_refresh_token()	Vérifie d'abord le cache disque (UserConfig) avant de redemander un token. Économise des appels réseau entre les redémarrages.
ensure_valid_token()	Appelé avant chaque fetch : renouvelle le token s'il expire dans moins de 60 secondes.
fetch_sap_logs()	GET vers MessageProcessingLogs avec \$select, \$orderby et \$top. Pour chaque log FAILED, lance un fetch concurrent de l'erreur complète.
fetch_packages()	GET vers IntegrationPackages. Parse les dates SAP au format /Date(timestamp)/ ou ISO.
fetch_artifacts()	GET vers IntegrationRuntimeArtifacts. Inclut le statut de déploiement.
fetch_all_package_artifacts()	Pour chaque packageId, récupère les DesigntimeArtifacts en parallèle (avec Semaphore pour limiter la concurrence).
fetch_artifact_properties()	Récupère les Configurations externalisées d'un artifact donné (paramètres de configuration).

Parsing des dates SAP

L'API SAP retourne les dates dans deux formats possibles : `/Date(1704067200000)/` (timestamp Unix en millisecondes) ou ISO 8601 (`2024-01-01T00:00:00`). La fonction `clean_sap_date()` gère les deux formats via une regex et un fallback.

5.3 config.rs — Configuration

Rôle : Deux responsabilités distinctes : la configuration SAP (chargée depuis les variables d'environnement) et la configuration utilisateur (persistée sur disque en TOML).

SapConfig

Charge `SAP_BASE_URL` , `SAP_TOKEN_URL` , `CLIENT_ID` et `CLIENT_SECRET` depuis l'environnement. Expose des méthodes pour construire les URLs des différents endpoints :

- `[object Object]` → `/api/v1/IntegrationPackages`
- `[object Object]` → `/api/v1/IntegrationRuntimeArtifacts`
- `[object Object]` → `/api/v1/MessageProcessingLogs` avec `$select/$orderby/$top/$filter`
- `[object Object]` → `ErrorInformation` d'un log
- `[object Object]` → `Configurations` d'un artifact

TokenCache

Wraps le token OAuth avec une date d'expiration (`Instant`). La méthode `is_expired()` renvoie `true` 60 secondes avant l'expiration réelle, pour permettre un renouvellement proactif.

UserConfig

Sérialisée/désérialisée via Serde + TOML. Stocke les préférences (limite de logs, parallélisme) et le cache du token OAuth (pour éviter une authentification à chaque démarrage). Sauvegardée automatiquement à la fermeture de l'application.

SapConfig a été étendu avec :

1. `tenant_id`: String — identifiant du tenant
2. `load_all_from_db()` — charge tous les tenants actifs depuis la table tenants, déchiffre les secrets via `crypto::decrypt()`
3. `from_env()` — conservé comme fallback pour compatibilité

5.4 crypto.rs — Chiffrement AES-256-GCM

Nouveau module dédié au chiffrement des secrets OAuth des tenants. Utilise AES-256-GCM avec nonce aléatoire de 12 bytes. Le résultat est encodé en base64 (nonce + ciphertext concaténés). La clé de chiffrement est lue depuis la variable d'environnement `TENANT_ENCRYPTION_KEY`.

Fonction	Rôle
<code>encrypt(plaintext)</code>	Chiffre un secret et retourne une chaîne base64
<code>decrypt(encoded)</code>	Déchiffre une chaîne base64 et retourne le plaintext

5.5 models.rs — Modèles de Données

Rôle : Définit toutes les structures de données utilisées pour la désérialisation JSON (via Serde) des réponses OData SAP, et les types internes partagés entre les couches.

Structures principales

LogEntry	Log d'exécution MPL. Champs: MessageGuid, Status, LogStart, IntegrationFlowName. Champs calculés (skip serde): parsed_date, error_message.
IntegrationPackage	Package SAP CPI avec métadonnées (id, name, version, vendor, dates, tags). Gestion des alias Serde pour les variantes de casse SAP.
RuntimeArtifact	Artifact déployé (iflow, mapping...) avec statut et date de déploiement.
DesigntimeArtifact	Artifact côté design-time (dans un package). Utilisé pour mapper les artifacts runtime à leur package.
ArtifactConfiguration	Paramètre externalisé d'un artifact (clé, valeur, type de données).
ArtifactError	Erreur de déploiement avec son timestamp.
Stats	Compteurs agrégés (total logs, logs FAILED, packages, artifacts) pour l'onglet Analytics.
RefreshData	Payload complet envoyé du worker de fetch à l'UI via le canal mpssc. Contient toutes les données pour l'affichage.

Ajout de AlertType — enum Rust représentant les types d'alertes intelligentes :

```
pub enum AlertType {
    Spike { recent_count: u64, avg_count: f64 },
    Regression,
    PersistentFailure { duration_mins: u64 },
}
```

5.6 db.rs — Accès à la Base de Données

Rôle : Toutes les opérations PostgreSQL via SQLx. Insertions en batch (QueryBuilder), mises à jour, requêtes de lecture pour l'UI et gestion de la table pending_alerts.

Pattern d'insertion — ON CONFLICT

Toutes les insertions utilisent `ON CONFLICT (...) DO UPDATE` (upsert). Cela garantit l'idempotence : on peut relancer le fetch autant de fois que nécessaire sans créer de doublons.

Fonctions clés

<code>insert_logs()</code>	Insertion batch des LogEntry. Upsert sur message_guid.
<code>insert_packages()</code>	Insertion batch des IntegrationPackage. Agrège les champs industries/keywords/products/countries/line_of_business en une colonne tags.
<code>insert_artifacts()</code>	Insertion batch des RuntimeArtifact. Upsert sur id.
<code>insert_artifact_error()</code>	Upsert d'une erreur de déploiement.
<code>insert_configurations()</code>	Batch d'ArtifactConfiguration. Upsert sur (artifact_id, parameter_key).
<code>bulk_update_artifact_package()</code>	Met à jour package_id pour un ensemble d'artifacts en une seule requête via UNNEST (très efficace pour les gros volumes).
<code>get_latest_log_date()</code>	Retourne le MAX(parsed_date) pour le fetch incrémental.
<code>insert_pending_alert()</code>	Insère une alerte en attente (ON CONFLICT DO NOTHING pour l'anti-doublon).
<code>fetch_pending_alerts()</code>	Lit toutes les alertes en attente triées par date.
<code>delete_pending_alerts()</code>	Supprime les alertes envoyées avec succès par leur ID.
<code>ensure_pending_alerts_table()</code>	CREATE TABLE IF NOT EXISTS au démarrage — évite une migration manuelle.

5.7 queries.rs — Requêtes Agrégées

Rôle : Construit le RefreshData complet en lançant toutes les requêtes de lecture en parallèle (tokio::join!) et en les assemblant dans une seule structure.

La fonction principale `build_refresh_data()` lance simultanément 13 requêtes indépendantes via `tokio::join!` : logs, exec_errors, artifacts, packages, deploy_errors, configurations, statistiques globales, erreurs par heure (sparkline 24h), erreurs par jour (barchart 7j), volume d'activité (sparkline 12h), top 5 artifacts en erreur, statuts logs, statuts artifacts.

Requêtes de visualisation

fetch_hourly_errors()	Erreurs FAILED groupées par heure sur 24h → données pour le Sparkline de l'onglet Analytics.
fetch_daily_errors()	Erreurs par jour sur 7 jours → données pour le BarChart hebdomadaire.
fetch_activity()	Volume total (toutes erreurs) par heure sur 12h → Sparkline d'activité globale.
fetch_top_errors()	Top 5 artifacts avec le plus d'entrées dans artifact_errors → BarChart "Top 5 Artifacts en Erreur".
fetch_log_statuses() + fetch_artifact_statuses()	Comptage par statut fusionné → tableau de répartition des statuts dans Analytics.

Fonction	Description
detect_spike()	Détecte les pics d'erreurs : nombre de FAILED sur 10 dernières minutes > 2x la moyenne historique. Cooldown : pas de détection si déjà signalé dans les 30 dernières minutes.
detect_regression()	Détecte les régressions via SQL ROW_NUMBER() OVER PARTITION : dernier log FAILED alors que l'avant-dernier ne l'était pas.
detect_persistent_failure()	Détecte les flux en échec prolongé (> 30 minutes sans succès) via EXTRACT(EPOCH FROM ...).

5.8 webhook.rs — Alertes Microsoft Teams

Rôle : Envoi de notifications vers Microsoft Teams sous forme d'Adaptive Cards (format MessageCard), avec un mécanisme anti-spam par groupement.

Flux de traitement

1. Lecture de toutes les pending_alerts depuis la base
2. Groupement par (flow_name, error_type) pour éviter les notifications redondantes
3. Construction d'une MessageCard Teams pour chaque groupe
4. POST HTTP vers WEBHOOK_URL
5. Suppression des alertes envoyées avec succès

Format des alertes

Chaque alerte Teams contient : le nom du flux concerné, le type d'erreur (exécution ou déploiement), le nombre d'occurrences, la date de première détection, un extrait de l'erreur (250 caractères), la liste des GUIDs concernés (max 5), et un bouton "Ouvrir SAP BTP".

Le worker est déclenché toutes les 60 secondes depuis `main.rs`. La variable d'environnement `WEBHOOK_URL` doit être définie pour activer les alertes.

Alertes intelligentes (smart_alerts)

4. Appel des 3 fonctions de détection : `detect_spike()`, `detect_regression()`, `detect_persistent_failure()`
5. Upsert dans `smart_alerts` via `upsert_smart_alert()`
6. Envoi d'une MessageCard Teams par type d'alerte, avec infos contextuelles (nb erreurs, durée, etc.)
7. Mise à jour du status à 'SENT' via `mark_smart_alerts_sent()`

Cooldown géré en SQL via `last_triggered_at` dans la table `smart_alerts`

5.9 ui.rs — Interface TUI Ratatui

Rôle : L'intégralité de l'interface utilisateur en mode terminal. Rendu Ratatui, gestion des événements clavier, navigation entre onglets, filtres, overlays modaux.

L'interface TUI a été refactorisée en module avec plusieurs fichiers pour la maintenabilité :

Fichier	Contenu
<code>ui/mod.rs</code>	Point d'entrée, <code>run_tui()</code> , <code>run_loop()</code> , <code>draw()</code> , gestion des événements clavier
<code>ui/app.rs</code>	Struct App, état complet, filtres (search + date + tenant), navigation
<code>ui/theme.rs</code>	Palette de couleurs, <code>status_style()</code> , <code>header_row()</code> , <code>table_block()</code> , <code>render_scrollbar()</code>
<code>ui/footer.rs</code>	<code>draw_header()</code> et <code>draw_footer()</code>
<code>ui/overlay.rs</code>	Overlays : <code>ArtifactDetail</code> , <code>LogDetail</code> , <code>Calendar</code> , <code>TenantFilter</code> , <code>Done/Error</code>
<code>ui/tabs/logs.rs</code>	<code>draw_logs_master_detail()</code> , <code>build_log_row()</code> , <code>log_detail_text()</code>
<code>ui/tabs/artifacts.rs</code>	<code>draw_artifacts_table()</code>
<code>ui/tabs/packages.rs</code>	<code>draw_packages_master_detail()</code>
<code>ui/tabs/errors.rs</code>	<code>draw_exec_errors_table()</code> , <code>draw_deploy_errors_table()</code>
<code>ui/tabs/analytics.rs</code>	<code>draw_analytics()</code> , <code>draw_stats_and_charts()</code>

5.10 utils.rs — Utilitaires

Rôle : Fonctions utilitaires partagées. Actuellement : création d'un spinner de progression CLI via la crate indicatif, utilisé uniquement pour le mode `--health`.

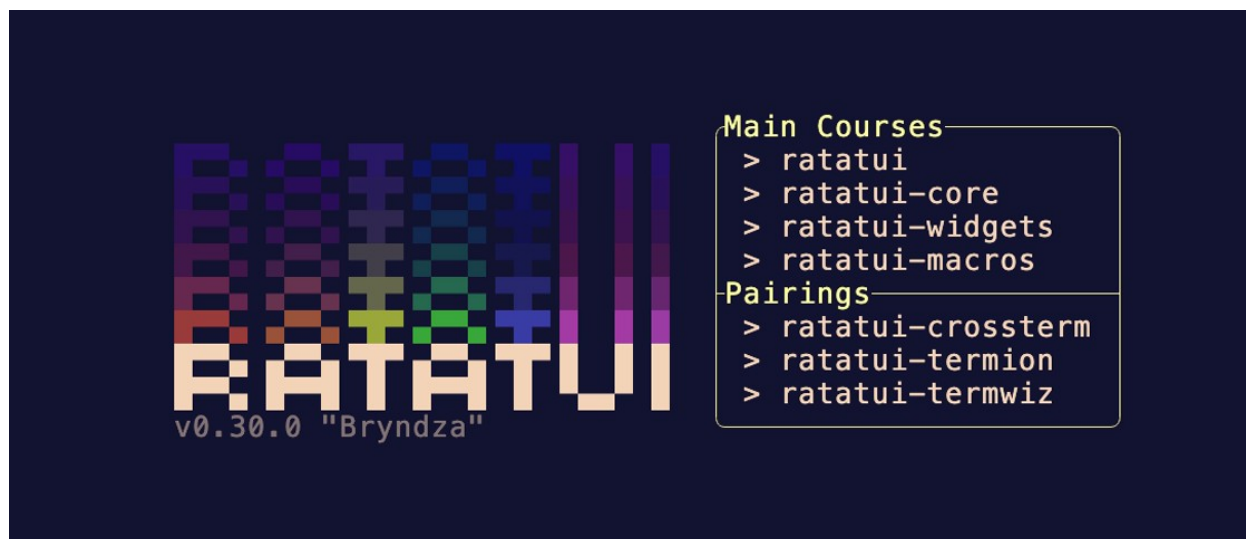
6. Guide de l'Interface TUI

⚠ Interface en cours de développement

L'interface TUI (Terminal User Interface) est fonctionnelle dans sa version actuelle mais continue d'évoluer. Certaines fonctionnalités peuvent changer entre les versions. Les captures d'écran et comportements décrits ici reflètent la version 0.3.0.

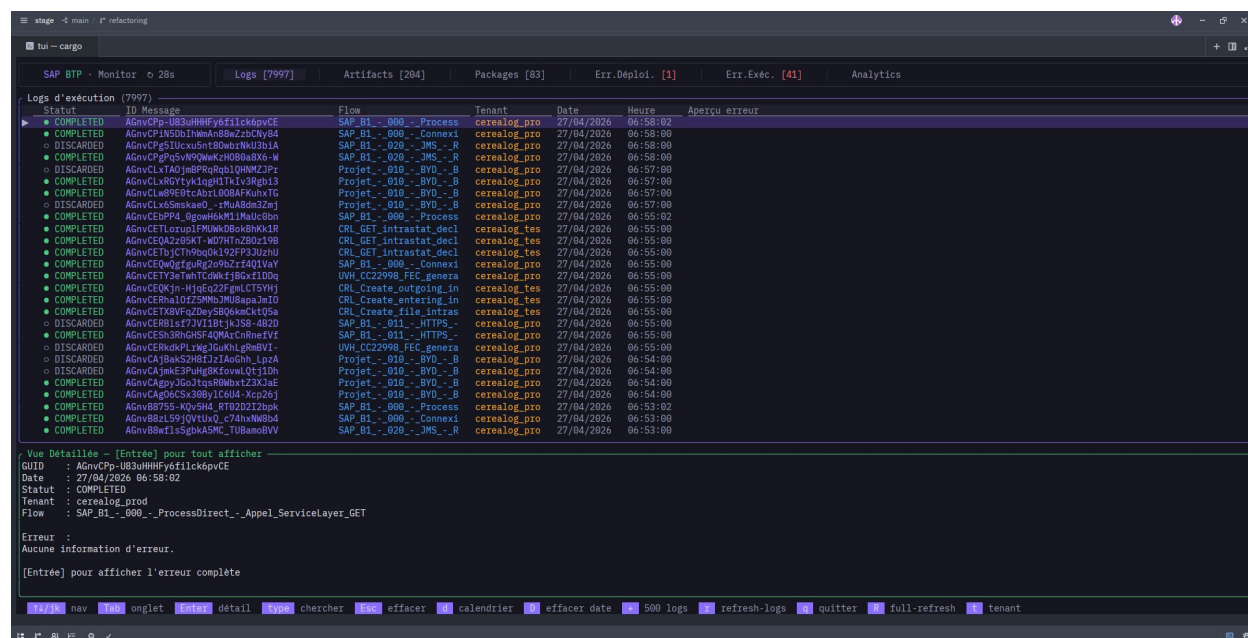
6.1 Présentation générale

L'interface TUI est construite avec Ratatui, une bibliothèque Rust de rendu terminal haute performance. Elle fonctionne en mode plein écran dans n'importe quel terminal compatible VT100 (iTerm2, Windows Terminal, Alacritty, tmux, etc.).



L'interface est organisée en trois zones :

- **Header (haut)** : Titre de l'application, indicateur de fraîcheur des données, spinner de rafraîchissement, filtre de date actif, et barre de navigation par onglets.
- **Body (centre)** : Contenu de l'onglet actif — tableau de données, détails, graphiques.
- **Footer (bas)** : Aide contextuelle avec tous les raccourcis clavier disponibles. En mode recherche, affiche la barre de saisie.



6.2 Navigation entre onglets

L'application dispose de 6 onglets, accessibles par Tab / Shift+Tab ou les flèches Gauche/Droite :

Onglet	Touche raccourci	Contenu
Logs	Tab (1er)	Tous les logs d'exécution SAP (MessageProcessingLogs), avec aperçu du détail en bas
Artifacts	Tab (2ème)	Liste des IntegrationRuntimeArtifacts avec statut de déploiement
Packages	Tab (3ème)	Integration Packages avec vue master-détail (artifacts + activité)
Err.Déploi.	Tab (4ème)	Erreurs de déploiement BTP (artifact_errors). Badge rouge si erreurs présentes
Err.Exéc.	Tab (5ème)	Logs FAILED uniquement, avec détail en bas. Badge rouge si erreurs présentes
Analytics	Tab (6ème)	Tableaux de bord : KPIs, sparklines, barcharts, répartition des statuts

Chaque onglet affiche un badge **[N]** indiquant le nombre total d'éléments (ou **[filtré/total]** si un filtre est actif). Les onglets Err.Déploi. et Err.Exéc. affichent leur badge en rouge si des erreurs sont présentes.

6.3 Navigation dans les listes

Touche	Action
↓ ou j	Ligne suivante
↑ ou k	Ligne précédente
Entrée	Afficher le détail / entrer dans un sous-panneau (onglet Packages)
Esc ou q	Quitter l'application (ou revenir au panneau principal dans l'onglet Packages)

6.4 Onglet Logs

Vue principale de l'application. Affiche les derniers N logs SAP (N = logs_limit).

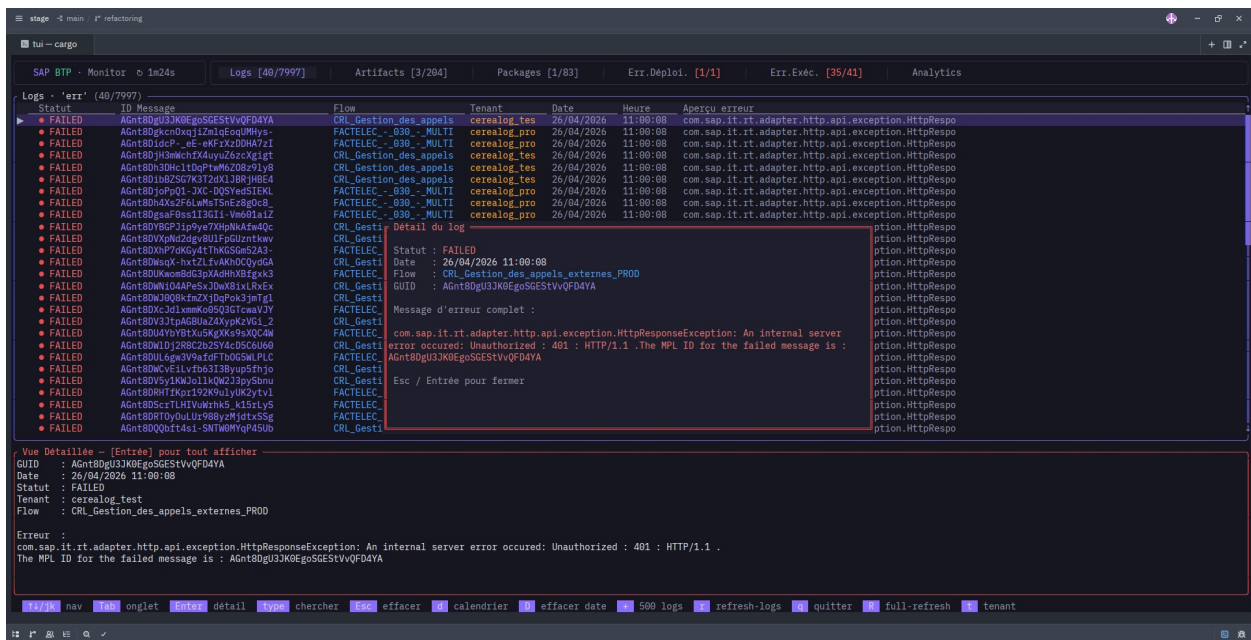
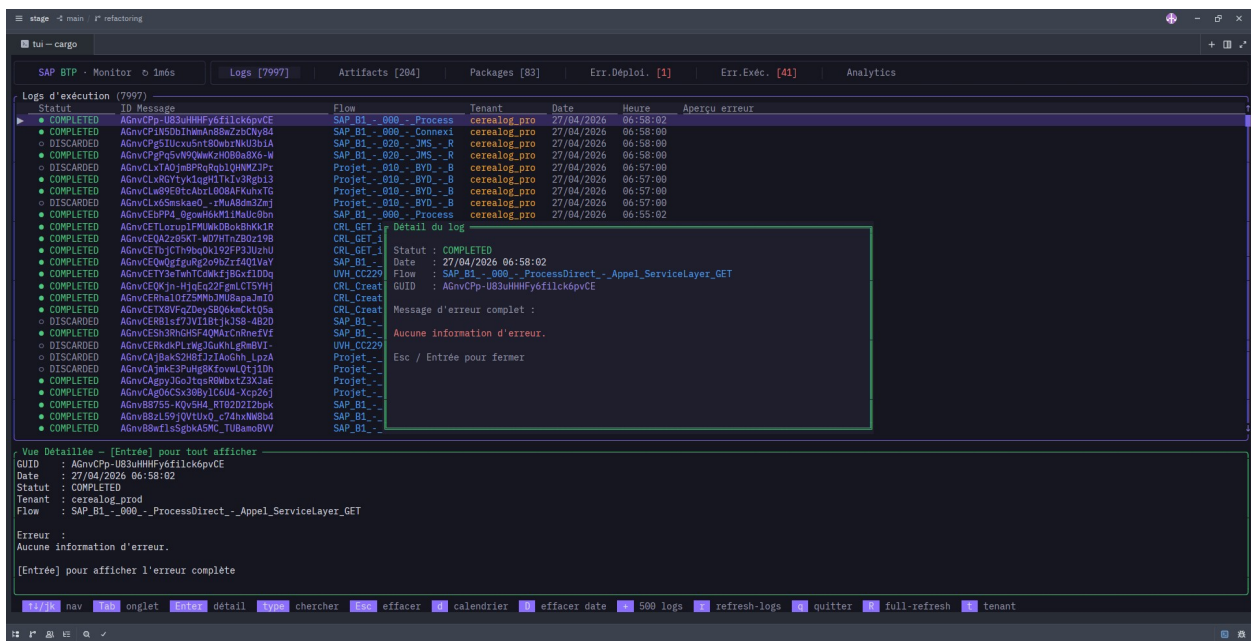
Colonnes affichées

- Statut : icône colorée (● vert = COMPLETED, ● rouge = FAILED, ○ orange = PROCESSING/STARTING, ● bleu = STARTED)
- ID Message : MessageGuid (identifiant unique SAP)
- Flow : Nom du flux d'intégration (IntegrationFlowName, tronqué à 22 caractères)
- Date + Heure : Date/heure d'exécution (LogStart parsée)
- Aperçu erreur : Extrait du message d'erreur (50 premiers caractères)
- Tenant : nom du tenant

Panneau de détail

Le tiers inférieur de l'écran affiche le détail du log sélectionné (GUID, date complète, statut, flux, aperçu de l'erreur). La couleur de la bordure reflète le statut (rouge/vert).

Entrée sur un log : ouvre un overlay modal avec l'erreur complète.



6.5 Onglet Artifacts

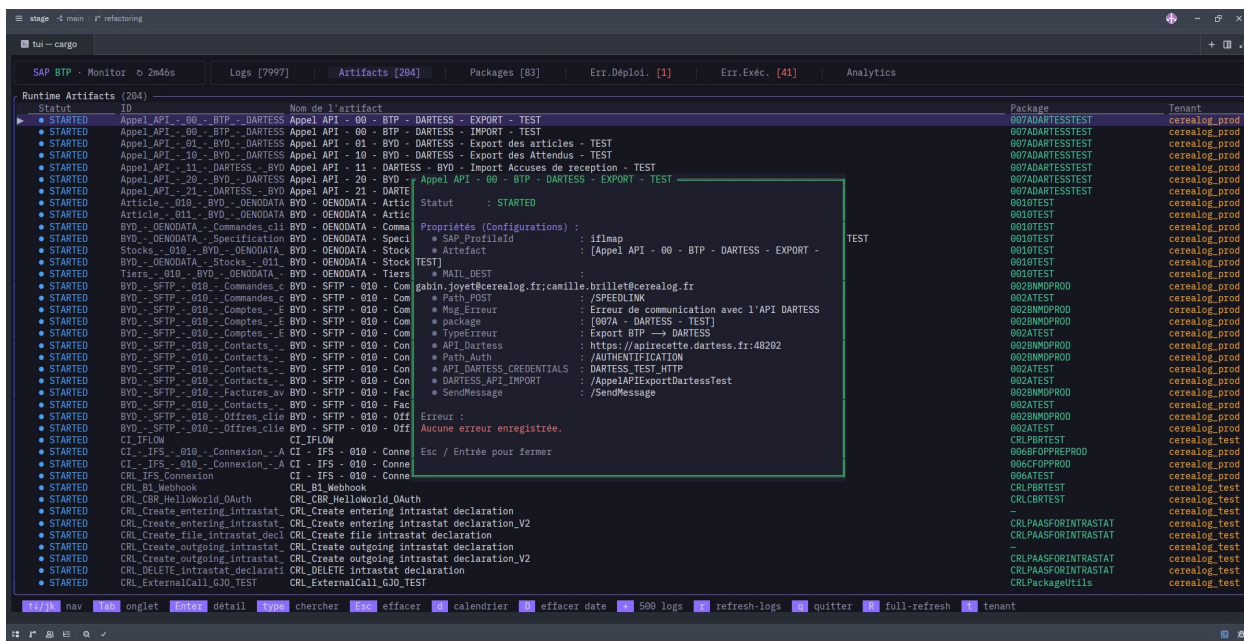
Liste tous les RuntimeArtifacts déployés sur la plateforme.

Statut	ID	Nom de l'artefact	Package	Tenant
STARTED	Appel_API_00_BTP_DARTESS	Appel API - 00 - BTP - DARTESS - EXPORT - TEST	007ADARTESSTEST	cerealog_prod
STARTED	Appel_API_00_BTP_DARTESS	Appel API - 00 - BTP - DARTESS - IMPORT - TEST	007ADARTESSTEST	cerealog_prod
STARTED	Appel_API_01_BYD_DARTESS	Appel API - 01 - BYD - DARTESS - Export des articles - TEST	007ADARTESSTEST	cerealog_prod
STARTED	Appel_API_10_BYD_DARTESS	Appel API - 10 - BYD - DARTESS - Export des Attendus - TEST	007ADARTESSTEST	cerealog_prod
STARTED	Appel_API_11_DARTESS_BYD	Appel API - 11 - DARTESS - BYD - Import Accusés de réception - TEST	007ADARTESSTEST	cerealog_prod
STARTED	Appel_API_20_BYD_DARTESS	Appel API - 20 - BYD - DARTESS - Export des commandes clients - TEST	007ADARTESSTEST	cerealog_prod
STARTED	Appel_API_21_DARTESS_BYD	Appel API - 21 - DARTESS - BYD - Import des avis dexpeditions	007ADARTESSTEST	cerealog_prod
STARTED	Article_010_BYD_OENODATA	Article - 010 - ProcessDirect - Export des articles BYD vers Oenodata - TEST	01010TEST	cerealog_prod
STARTED	Article_011_BYD_OENODATA	Article - 011 - Export en masse des articles BYD vers Oenodata - TEST	01010TEST	cerealog_prod
STARTED	BYD_OENODATA_Commandes_cli	BYD - OENODATA - Commandes clients - 010 - Export des commandes clients BYD vers Oenodata - TEST	01010TEST	cerealog_prod
STARTED	BYD_OENODATA_Specification	BYD - OENODATA - Specification de produits - 010 - ProcessDirect - Export des SOP BYD vers Oenodata - TEST	01010TEST	cerealog_prod
STARTED	Stocks_010_BYD_OENODATA	Stocks - 010 - Export des stocks BYD vers Oenodata - TEST	01010TEST	cerealog_prod
STARTED	BYD_OENODATA_Stocks_011	BYD - OENODATA - Stocks - 011 - Export des stocks BYD vers Oenodata - TEST	01010TEST	cerealog_prod
STARTED	Tiers_010_BYD_OENODATA	Tiers - 010 - Export des clients - fournisseurs BYD vers Oenodata - TEST	01010TEST	cerealog_prod
STARTED	BYD_SFTP_010_Commandes_c	BYD - SFTP - 010 - Commandes clients - Export des commandes clients vers Salesforce - PROD	002BNMOPPROD	cerealog_prod
STARTED	BYD_SFTP_010_Commandes_c	BYD - SFTP - 010 - Commandes clients - Export des commandes clients vers Salesforce - TEST	002ATTEST	cerealog_prod
STARTED	BYD_SFTP_010_Comptes_E	BYD - SFTP - 010 - Comptes - Export des comptes vers Salesforce - PROD	002BNMOPPROD	cerealog_prod
STARTED	BYD_SFTP_010_Comptes_E	BYD - SFTP - 010 - Comptes - Export des comptes vers Salesforce - PROD	002BNMOPPROD	cerealog_prod
STARTED	BYD_SFTP_010_Comptes_E	BYD - SFTP - 010 - Comptes - Export des comptes vers Salesforce - TEST	002ATTEST	cerealog_prod
STARTED	BYD_SFTP_010_Contacts	BYD - SFTP - 010 - Contacts - Export des contacts vers Salesforce - PROD	002BNMOPPROD	cerealog_prod
STARTED	BYD_SFTP_010_Contacts	BYD - SFTP - 010 - Contacts - Export des contacts vers Salesforce - PROD	002BNMOPPROD	cerealog_prod
STARTED	BYD_SFTP_010_Contacts	BYD - SFTP - 010 - Contacts - Export des contacts vers Salesforce - TEST	002ATTEST	cerealog_prod
STARTED	BYD_SFTP_010_Contacts	BYD - SFTP - 010 - Contacts - Export des contacts vers Salesforce - TEST	002BNMOPPROD	cerealog_prod
STARTED	BYD_SFTP_010_Factures_av	BYD - SFTP - 010 - Factures avoirs - Export des ventes par periode vers Salesforce - PROD	002ATTEST	cerealog_prod
STARTED	BYD_SFTP_010_Factures_av	BYD - SFTP - 010 - Factures avoirs - Export des ventes par periode vers Salesforce - TEST	002BNMOPPROD	cerealog_prod
STARTED	BYD_SFTP_010_Offres_cli	BYD - SFTP - 010 - Offres clients - Export des offres clients vers Salesforce - PROD	002BNMOPPROD	cerealog_prod
STARTED	BYD_SFTP_010_Offres_cli	BYD - SFTP - 010 - Offres clients - Export des offres clients vers Salesforce - TEST	002ATTEST	cerealog_prod
STARTED	CI_FLOW	CI - FLOW	006BFOPPREPROD	cerealog_test
STARTED	CI_IPFS_010_Connexion_A	CI - IPFS - 010 - Connexion - Authentification - PREPROD	006BFOPPREPROD	cerealog_test
STARTED	CI_IPFS_010_Connexion_A	CI - IPFS - 010 - Connexion - Authentification - PROD	006BFOPPREPROD	cerealog_test
STARTED	CRL_IPS_Connexion	CI - IPS - 010 - Connexion - Authentification - TEST	006BFOPPREPROD	cerealog_test
STARTED	CRL_HelloWorld_OAuth	CRL_HelloWorld_OAuth	CRLCBRTTEST	cerealog_test
STARTED	CRL_Create_entering_intrastat	CRL_Create entering intrastat declaration	-	cerealog_test
STARTED	CRL_Create_entering_intrastat	CRL_Create entering intrastat declaration_V2	CRLPAASFORINTRASTAT	cerealog_test
STARTED	CRL_Create_file_intrastat_decl	CRL_Create file intrastat declaration	CRLPAASFORINTRASTAT	cerealog_test
STARTED	CRL_Create_outgoing_intrastat	CRL_Create outgoing intrastat declaration	-	cerealog_test
STARTED	CRL_Create_outgoing_intrastat	CRL_Create outgoing intrastat declaration_V2	CRLPAASFORINTRASTAT	cerealog_test
STARTED	CRL_DELETE_intrastat_declarati	CRL_DELETE intrastat declaration	CRLPAASFORINTRASTAT	cerealog_test
STARTED	CRL_ExternalCall_G30_TEST	CRL_ExternalCall_G30_TEST	CRLPackageutils	cerealog_test

Colonnes affichées

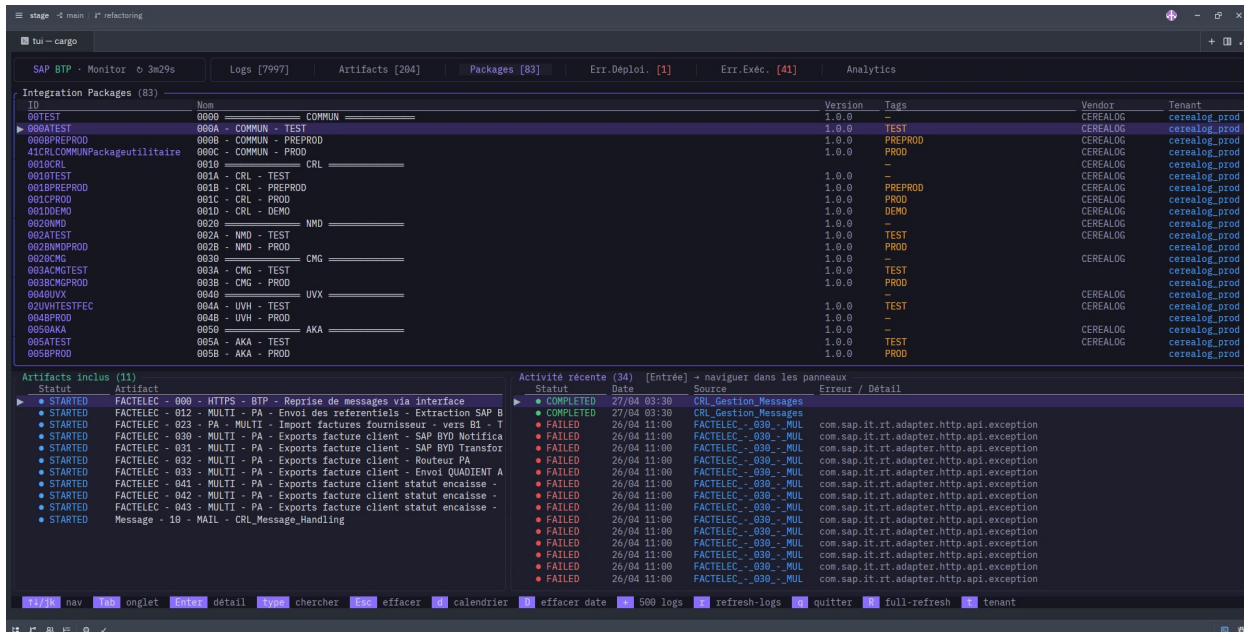
- Statut : STARTED (vert/bleu), ERROR (rouge), etc.
- ID : Identifiant technique SAP de l'artefact
- Nom de l'artefact : Nom fonctionnel
- Package : ID du package parent
- Tenant : nom du tenant

Entrée sur un artefact : ouvre un overlay modal avec le statut, les configurations externalisées (paramètres clé/valeur) et le message d'erreur de déploiement si applicable.



6.6 Onglet Packages

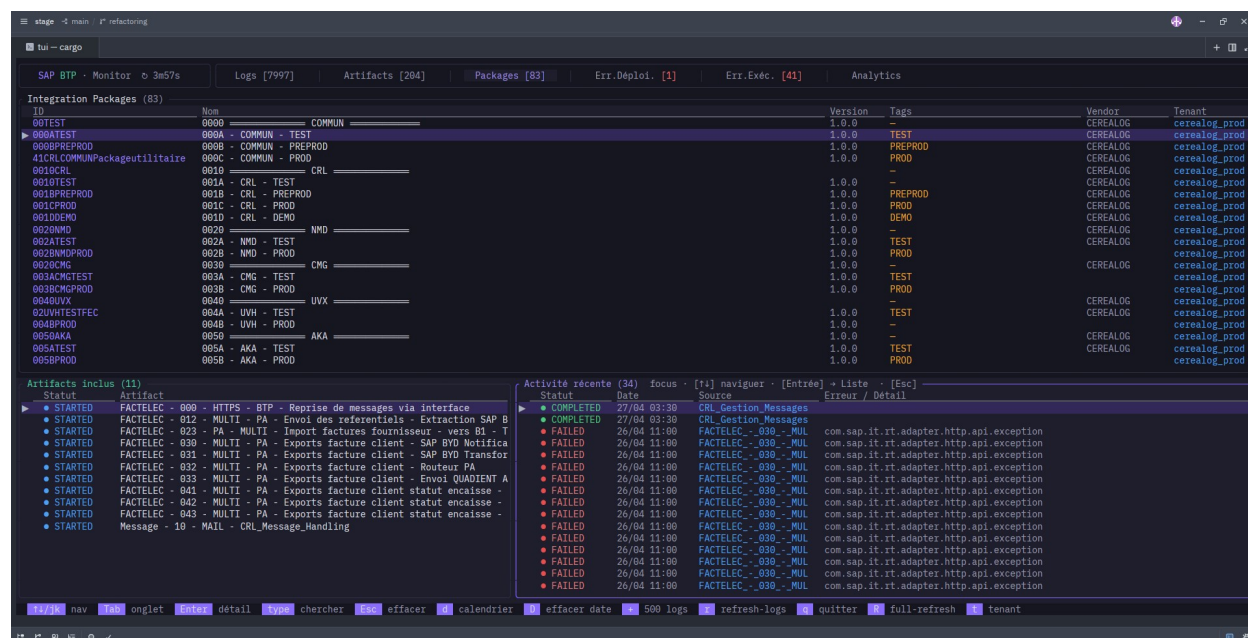
Vue master-détail avec navigation à trois niveaux.



Niveaux de navigation

- Liste principale (focus par défaut)** : tous les Integration Packages avec ID, nom, version, tags, vendor.
- Panneau Artifacts (Entrée depuis la liste)** : artifacts inclus dans le package sélectionné, avec leur statut.
- Panneau Activité (Entrée depuis les artifacts)** : logs d'exécution récents des artifacts du package (50 derniers). Retour à la liste : Entrée ou Esc.

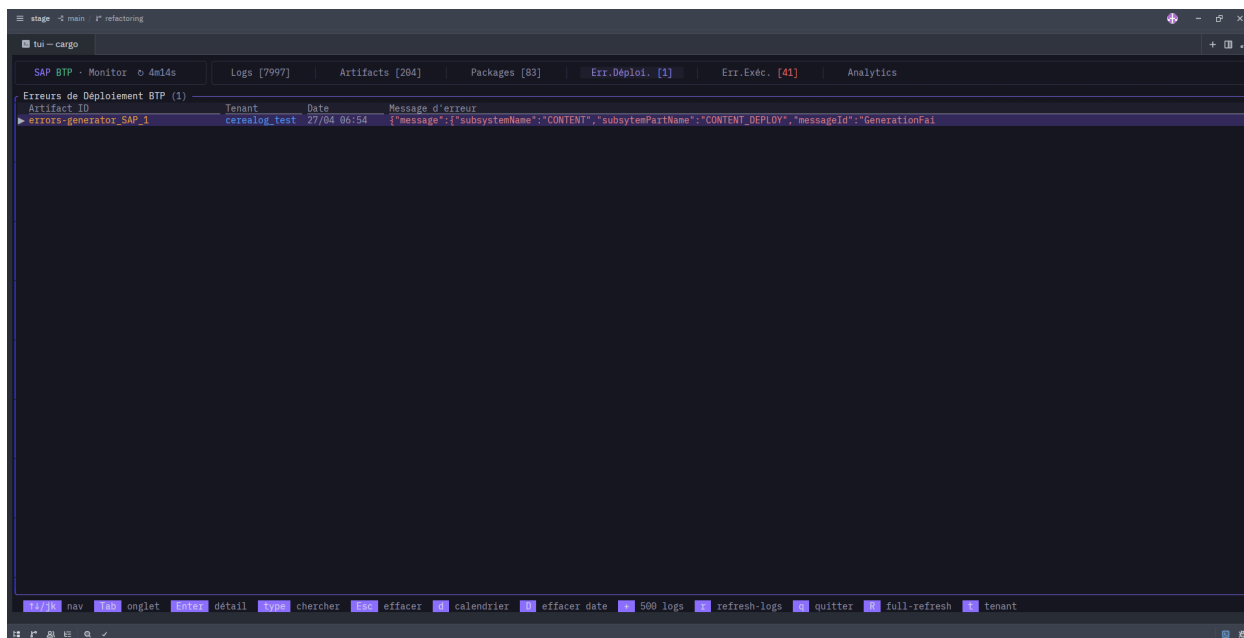
La couleur de la bordure du panneau actif passe en violet pour indiquer le focus courant.



6.7 Onglets Erreurs

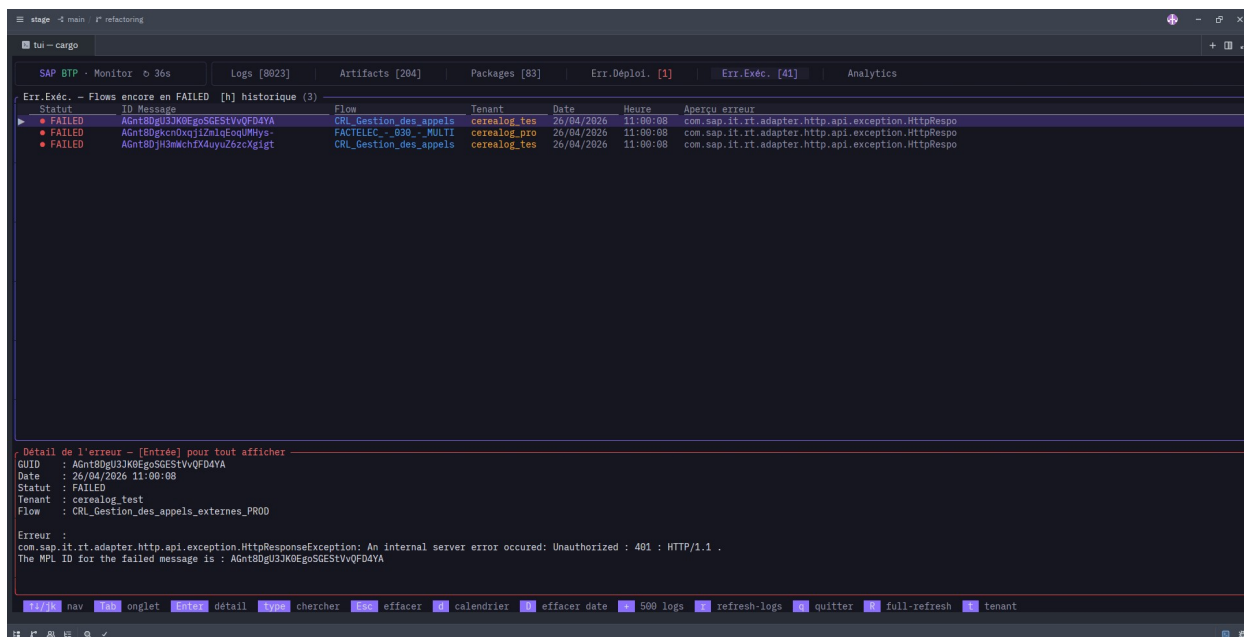
Err.Déploi. — Erreurs de Déploiement

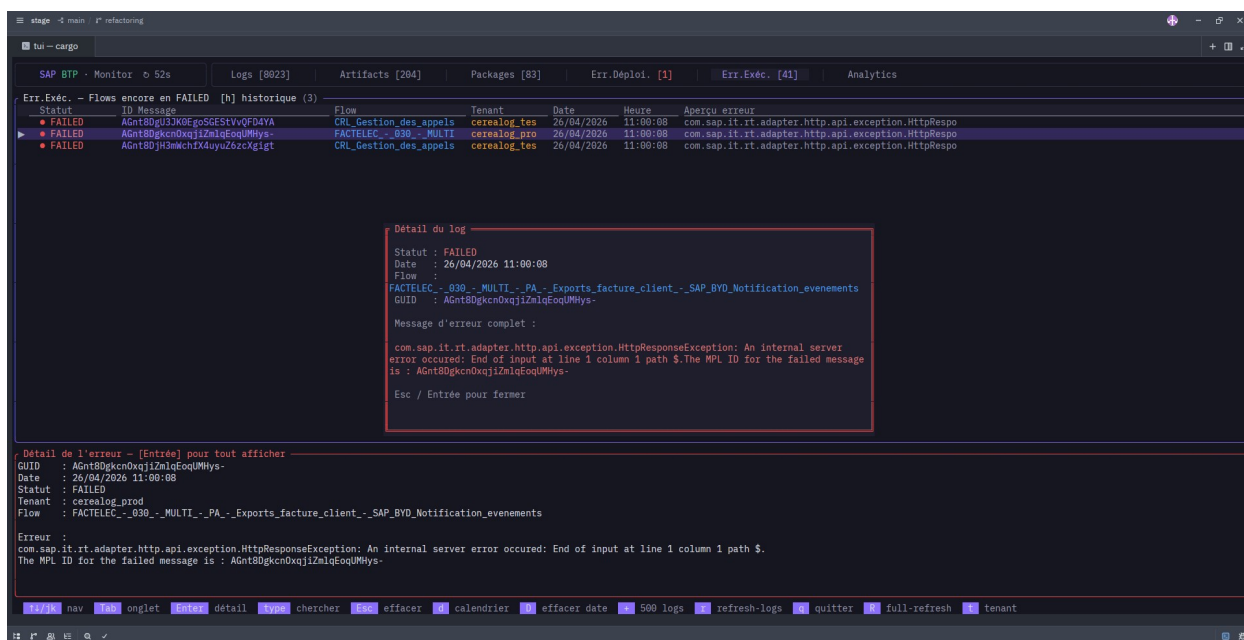
Liste les artifacts dont le déploiement a échoué, avec l'heure de l'erreur et un extrait du message. Si aucune erreur n'est présente, affiche un message de succès vert.



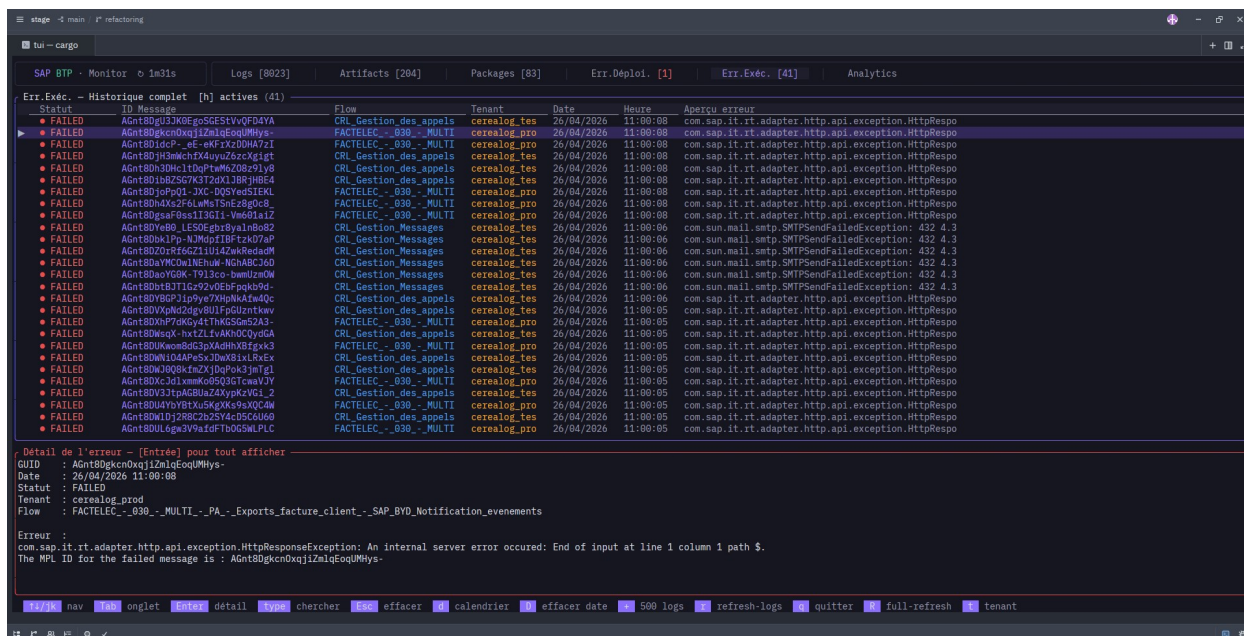
Err.Exéc. — Erreurs d'Exécution

Filtre automatiquement les logs FAILED uniquement. Même structure que l'onglet Logs (tableau + panneau de détail en bas). Idéal pour le monitoring des flux en production. Les erreurs affichées par défaut sont les erreurs en cours



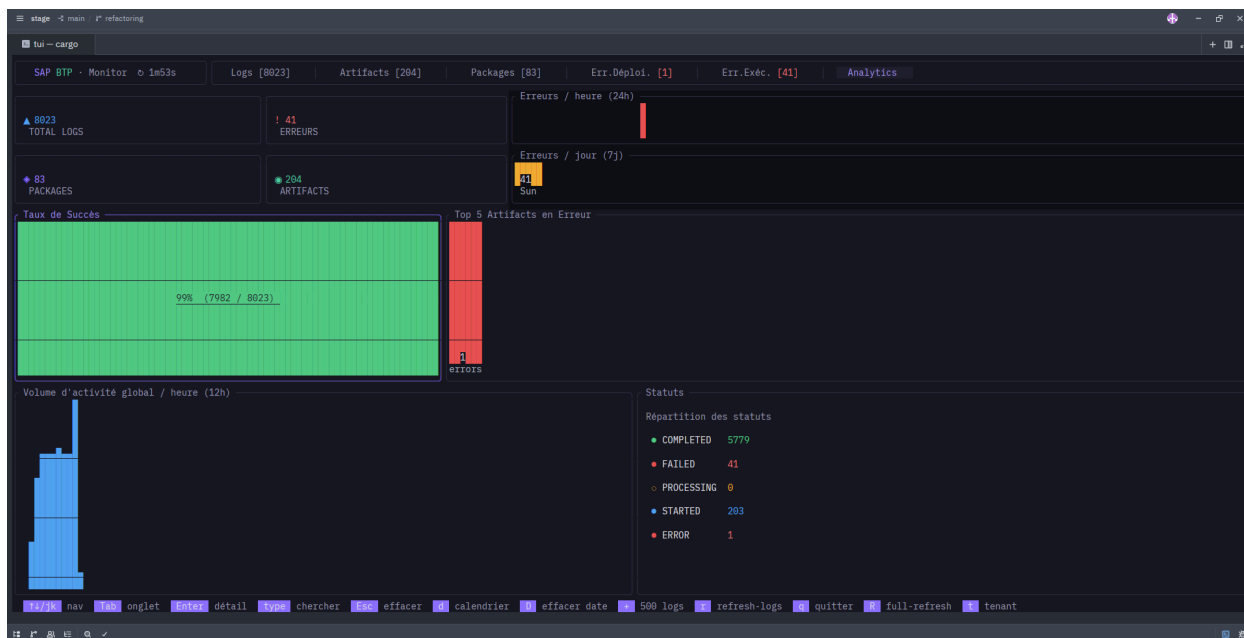


pour voir toutes les erreurs all-time, appuyer sur h :



6.8 Onglet Analytics

Tableau de bord visuel composé de 4 zones :

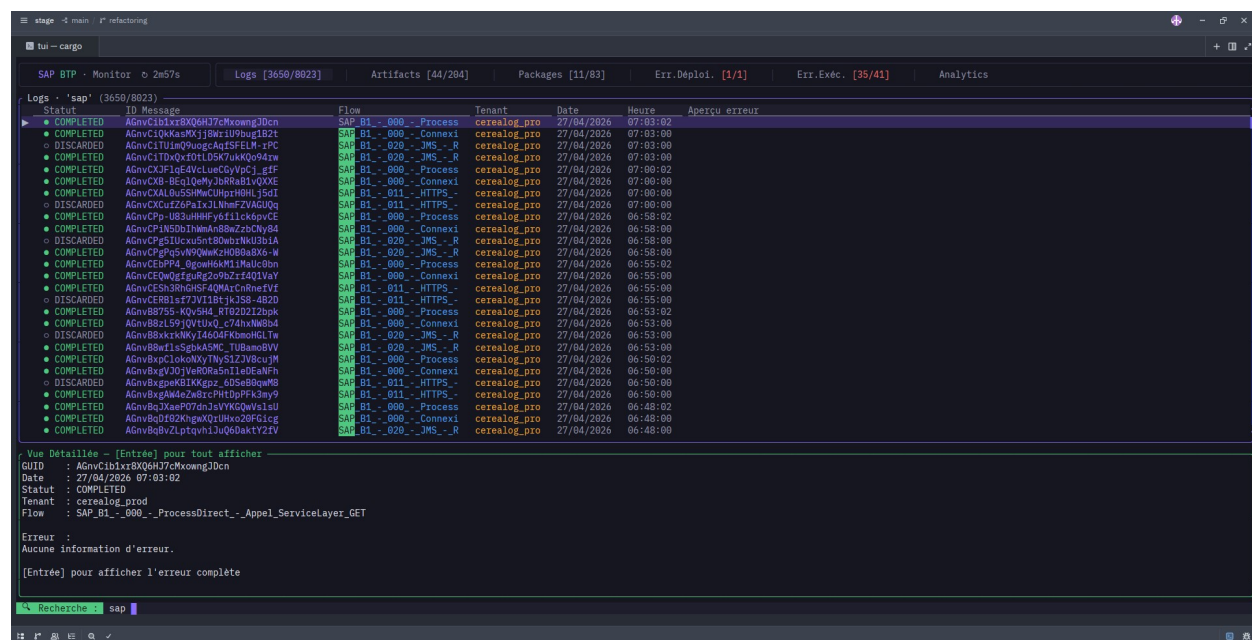


- **4 KPI Cards (gauche)** : Total Logs, Erreurs, Packages, Artifacts — mise à jour à chaque rafraîchissement.
- **Sparkline Erreurs/heure (droite haut)** : Graphique linéaire des erreurs FAILED sur les 24 dernières heures, par heure.
- **BarChart Erreurs/jour (droite bas)** : Histogramme des erreurs sur les 7 derniers jours.
- **Jauge de succès (bas gauche)** : Taux de succès global (COMPLETED / total). Vert > 90%, orange > 70%, rouge sinon.
- **Top 5 Artifacts en erreur (bas droite)** : BarChart des artifacts avec le plus d'erreurs de déploiement.
- **Activité globale / heure (bas)** : Sparkline du volume de messages sur 12h.
- **Répartition des statuts** : Comptage par statut (COMPLETED, FAILED, PROCESSING, STARTED, ERROR) fusionné logs + artifacts.

6.9 Recherche et Filtres

Recherche textuelle

Sur n'importe quel onglet (sauf Analytics), taper n'importe quelle lettre active le mode recherche. La barre de saisie s'affiche en bas à la place des raccourcis. Le filtre s'applique en temps réel sur tous les champs visibles.



RECHERCHE AVANCE :

Le système de recherche avancée permet de filtrer efficacement les données du TUI (logs, artifacts, packages...) avec :

- une **syntaxe expressive**
- des **alias intelligents**
- une **recherche multi-champs**
- un **respect strict de l'ordre chronologique**

La recherche repose sur **2 types de tokens** :

Filtres structurés (key : value)

status:FAILED
 tenant:cerealog
 flow:invoice-flow

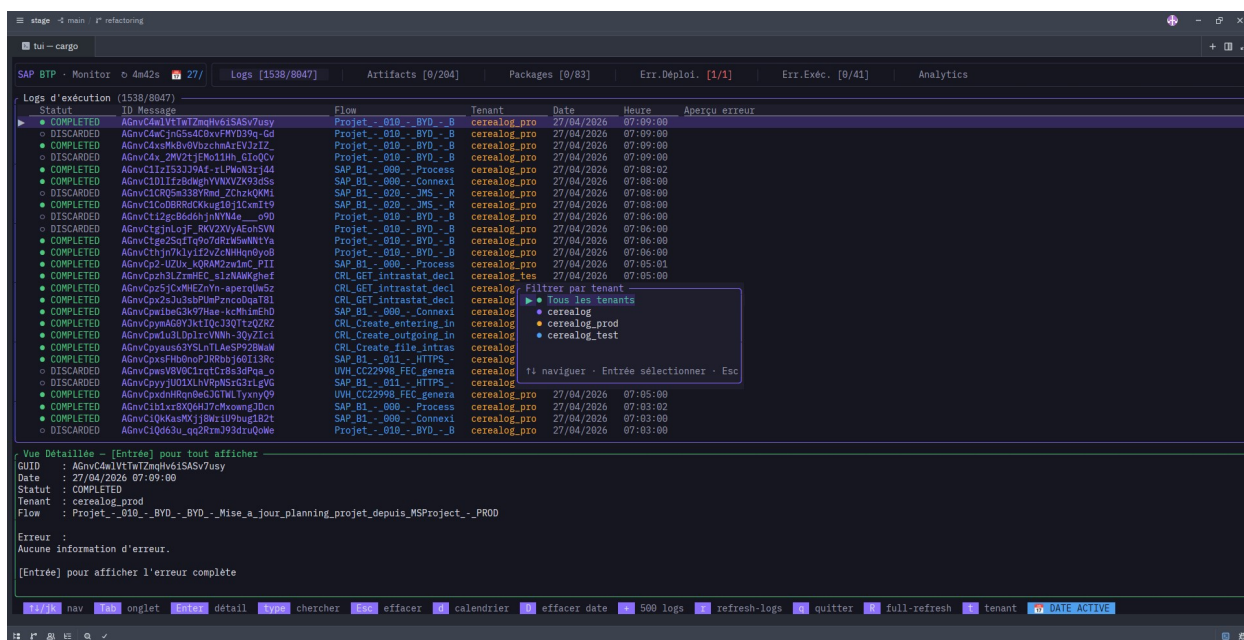
Recherche libre (full-text)

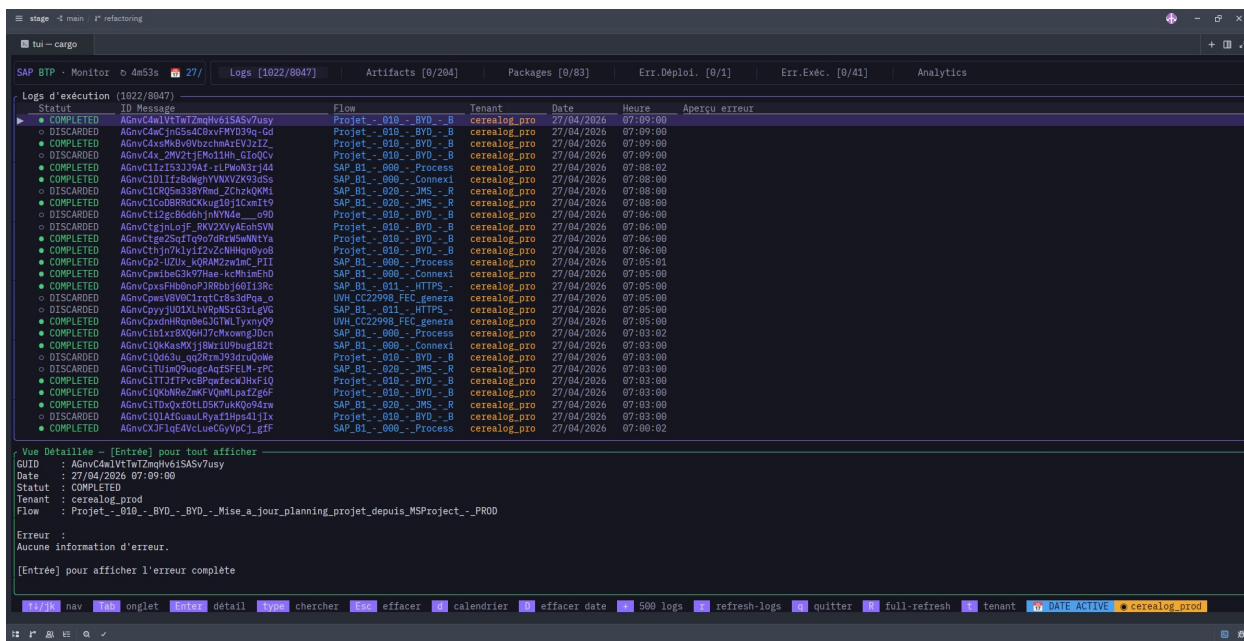
timeout error payment

Les deux peuvent être combinés :

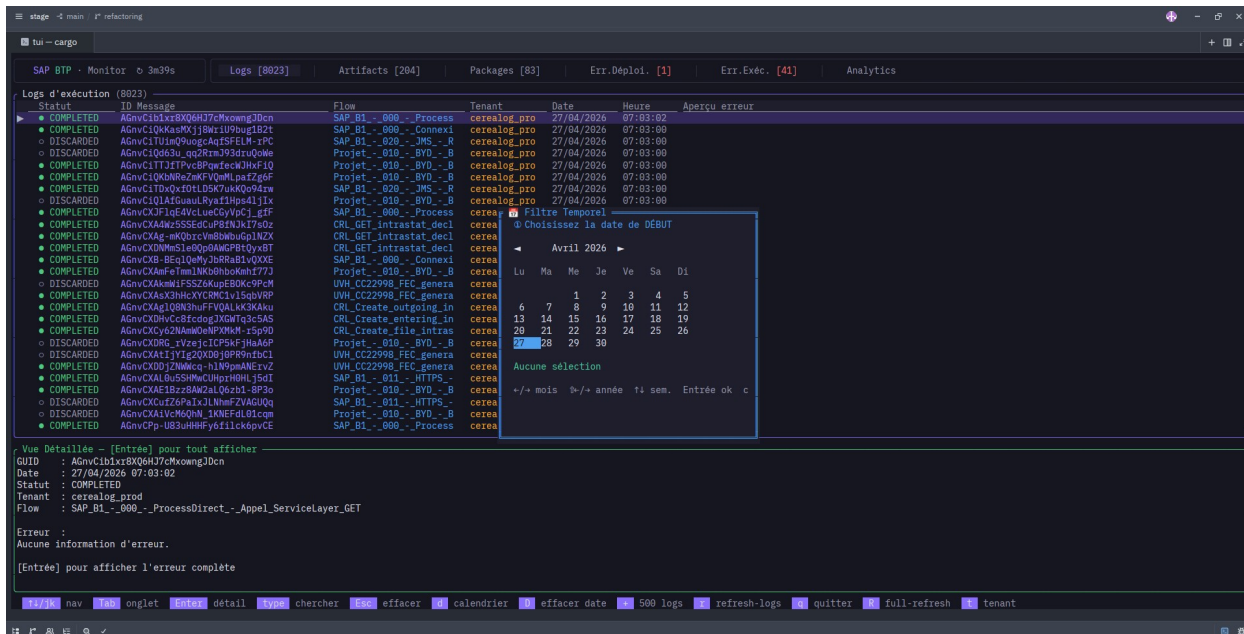
status:FAILED tenant:cerealog timeout

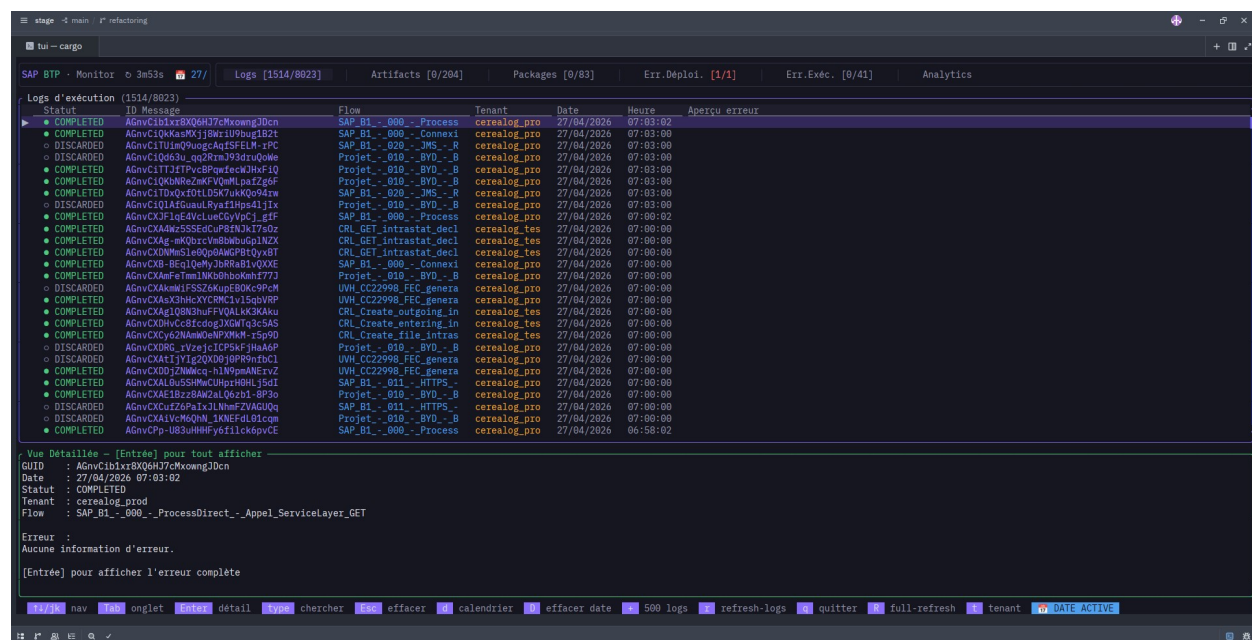
filtre par tenant





Filtre par date (Calendrier interactif)





La touche **d** ouvre le calendrier interactif pour filtrer par plage de dates. La sélection se fait en deux étapes (date début puis date fin). Pour annuler le filtrage par date **maj + d** :

Touche (calendrier)	Action
← / →	Déplacer le curseur d'un jour
↑ / ↓ ou k / j	Déplacer le curseur d'une semaine
Shift+ ← / Shift+ →	Mois précédent / suivant
Entrée	Confirmer la date (phase 1 = début, phase 2 = fin)
M (maj + m) ou c	Effacer le filtre de date
Esc	Fermer le calendrier sans appliquer

La touche **D** (majuscule) efface directement le filtre de date actif depuis n'importe quel onglet.

6.10 Rafraîchissement des données

Touche	Action
r	Refresh incrémental : récupère uniquement les nouveaux logs depuis le dernier fetch (plus rapide) (encore en développement + optimisation)
R	Full refresh : récupère l'intégralité des packages, artifacts et logs (plus lent, mais complet : environ 8 seconds, le goulot viens de l'attente de la réponse API SAP)
+	Augmente la limite de logs de 500 et relance un fetch complet (encore en développement + optimisation)

Un rafraîchissement automatique est déclenché toutes les 5 minutes (295 secondes) si l'application est inactive. L'indicateur de fraîcheur dans le header passe en rouge après 5 minutes sans données fraîches.

7. Dépendances du Projet

Le projet utilise les crates suivantes, toutes disponibles sur crates.io :

Crate	Version	Rôle
tokio	1	Runtime asynchrone — cœur du système concurrent. Features: full (scheduler multi-thread, timers, channels).
reqwest	0.11	Client HTTP async basé sur tokio. TLS via rustls. Features: json (sérialisation auto), rustls-tls.
serde	1.0	Framework de sérialisation/désérialisation. Feature derive génère automatiquement les impls via macros.
serde_json	1.0	Sérialisation JSON pour serde. Utilisé pour les payloads webhook Teams.
anyhow	1.0	Gestion d'erreurs ergonomique. Le type Result<T> unifie toutes les erreurs avec contexte automatique.
log + env_logger	0.4 / 0.10	Framework de logging structuré. Niveau configurable via RUST_LOG=info/debug/warn.
dotenvy	0.15	Chargement du fichier .env au démarrage.
chrono	0.4	Manipulation de dates et heures. Feature clock pour l'heure locale.
regex	1.10	Expressions régulières — utilisé pour parser les dates SAP /Date(ms)/.
sqlx	0.7	ORM async PostgreSQL. Queries compilées à la compilation, mapping via FromRow.

clap	4.4	Parsing des arguments CLI. Feature derive génère les structs d'args automatiquement.
indicatif	0.17	Barres de progression et spinners CLI pour le mode --health.
ratatui	0.26	Bibliothèque TUI — rendu de l'interface terminal complète.
crossterm	0.27	Backend terminal cross-platform pour ratatui (événements clavier, mode raw).
futures	0.3	Combinateurs de futures async — stream::buffer_unordered pour la concurrence limitée.
dirs	5	Chemins de configuration système (XDG sur Linux, AppData sur Windows, Library sur macOS).
toml	0.8	Sérialisation/désérialisation TOML pour la config utilisateur.
indexmap	=2.2.6	HashMap avec ordre d'insertion préservé (version épinglée pour compatibilité).

8. Référence Rapide

8.1 Raccourcis clavier — Résumé complet

Touche	Contexte	Action
Tab / Shift+Tab	Partout	Onglet suivant / précédent
← / →	Hors Packages (focus liste)	Onglet précédent / suivant
↓ / j	Dans une liste	Ligne suivante
↑ / k	Dans une liste	Ligne précédente
Entrée	Logs / ExecErrors	Overlay : erreur complète du log
Entrée	Artifacts	Overlay : configs + erreur de l'artefact
Entrée	Packages	Naviguer liste → artifacts → activité
r	Partout (hors refresh)	Refresh incrémental (logs uniquement)
R	Partout (hors refresh)	Full refresh (packages + artifacts + logs)
+	Partout	Ajouter 500 logs à la limite et recharger
d	Partout	Ouvrir le calendrier de filtre par date
D	Partout	Effacer le filtre de date actif
Toute lettre	Hors Analytics, hors mode recherche	Active la recherche et ajoute le caractère
Backspace	Mode recherche	Supprimer le dernier caractère
Esc	Mode recherche	Effacer la recherche
Esc / q	Hors overlay	Quitter l'application
Ctrl+C	Partout	Quitter immédiatement
t	Partout (hors analytics)	Filtrer le tenant

8.2 Variables d'environnement

Variable	Obligatoire	Description
SAP_BASE_URL	Oui	URL de base de l'API SAP Cloud Integration
SAP_TOKEN_URL	Oui	URL OAuth avec grant_type=client_credentials
CLIENT_ID	Oui	Client ID de l'instance de service SAP BTP
CLIENT_SECRET	Oui	Client Secret de l'instance de service SAP BTP
DATABASE_URL	Oui	Connection string PostgreSQL
WEBHOOK_URL	Non	URL webhook Microsoft Teams (alertes désactivées si absent)
RUST_LOG	Non	Niveau de log: error/warn/info/debug (défaut: error)

8.3 Commandes Cargo essentielles

cargo build	Compilation en mode debug
cargo build --release	Compilation optimisée (production)
cargo run	Compile et lance l'application (mode debug)
cargo run -- --health	Lance uniquement le health check
cargo run -- --full	Lance avec full refresh forcé
cargo run -- --top 1000	Lance avec limite de 1000 logs
cargo check	Vérifie les erreurs de compilation sans produire de binaire (très rapide)
cargo clippy	Linter Rust — analyse statique et suggestions d'amélioration
cargo fmt	Formatage automatique du code selon les conventions Rust
cargo update	Met à jour les dépendances vers leurs dernières versions compatibles
cargo doc --open	Génère et ouvre la documentation HTML du projet

9. Résolution des Problèmes Courants

9.1 Erreurs de compilation

error[E0433]: failed to resolve	Import manquant. Vérifier les use en haut du fichier concerné.
error[E0308]: mismatched types	Type incorrect. Lire le message du compilateur — il indique exactement le type attendu et reçu.
cannot find crate	Dépendance manquante dans Cargo.toml, ou cargo update nécessaire.
linker error (ld: library not found)	Sur macOS : installer Xcode Command Line Tools (xcode-select --install). Sur Linux : installer build-essential.

9.2 Erreurs d'exécution

SAP_BASE_URL manquante	Le fichier .env est absent ou mal placé (doit être à la racine du projet).
401 Unauthorized SAP	CLIENT_ID ou CLIENT_SECRET incorrect, ou token expiré. Vérifier dans la console SAP BTP.
DATABASE_URL manquante	Variable d'environnement absente. Vérifier le .env et que dotenvy le charge bien.
connection refused (PostgreSQL)	PostgreSQL n'est pas démarré ou écoute sur un autre port.
relation does not exist	Les tables SQL n'ont pas été créées. Exécuter le DDL de la section 3.4.
Terminal trop petit	L'interface TUI nécessite un terminal d'au moins 80x24 caractères.

9.3 Performances (encore en optimisation)

- Si le fetch initial est lent : augmenter la concurrence dans main.rs (log_concurrency et concurrency). Attention aux limites de rate de l'API SAP.
- Si l'UI est lente à s'afficher : utiliser le build release (cargo build --release). Le mode debug est non-optimisé.
- Si la base de données est lente : ajouter des index sur parsed_date et status dans sap_monitoring_logs.

10. Roadmap et Évolutions Prévues

Note de développement

Cette section liste les fonctionnalités envisagées pour les prochaines versions. Elles ne sont pas encore implémentées. Les priorités peuvent évoluer selon les retours des utilisateurs.

10.1 Fonctionnalités prioritaires

- 9. Tests automatisés (tests unitaires et d'intégration avec cargo test)
- 10. Migrations SQL automatisées via SQLx migrate (plus besoin de DDL manuel)
- 11. Export CSV/JSON des logs et artifacts depuis l'interface TUI
- 12. Pagination serveur-side pour les très gros volumes de logs (> 10 000)

10.2 Améliorations TUI

- 13. Tri des colonnes par clic (simulé via touches)
- 14. Vue graphique de la dépendance package → artifact → log
- 15. Raccourcis de navigation rapide (goto ligne, goto onglet)

10.3 Infrastructure

- 16. Support Docker (Dockerfile + docker-compose avec PostgreSQL)
- 17. CI/CD GitHub Actions (build, tests, clippy)

Glossaire

SAP BTP	SAP Business Technology Platform — plateforme cloud d'applications et d'intégrations SAP.
SAP CPI	SAP Cloud Platform Integration (aussi appelé SAP Integration Suite) — service d'intégration SAP hébergé dans BTP.
MPL	MessageProcessingLog — log d'exécution d'un message dans SAP CPI. Chaque exécution d'un flux génère un MPL.
iFlow	Integration Flow — flux d'intégration SAP, l'unité de base de traitement de messages dans CPI.
OData	Open Data Protocol — protocole REST/JSON standard de SAP pour exposer ses APIs.
TUI	Terminal User Interface — interface utilisateur en mode texte dans un terminal.
OAuth 2.0	Standard d'autorisation utilisé par SAP BTP pour sécuriser l'accès à ses APIs.
Crate	Bibliothèque ou package Rust, équivalent d'un module npm ou d'un package Python.
Cargo	Gestionnaire de paquets et outil de build officiel de Rust.
rustup	Gestionnaire de versions de la toolchain Rust (compilateur, bibliothèque standard, etc.).
Tokio	Runtime asynchrone Rust le plus utilisé. Fournit le scheduler, les timers et les primitives de synchronisation.
SQLx	Bibliothèque Rust de base de données async avec vérification des requêtes SQL à la compilation.
Ratatui	Bibliothèque Rust pour créer des interfaces TUI richement formatées dans le terminal.
Upsert	INSERT ... ON CONFLICT ... DO UPDATE — opération d'insertion idempotente (crée ou met à jour).
Adaptive Card	Format de carte enrichie Microsoft (JSON) supporté par Teams, Outlook, etc.
Debouncing	Technique anti-spam : regrouper plusieurs événements identiques avant de les traiter.